*El hamouni Anisse **n°12311622***

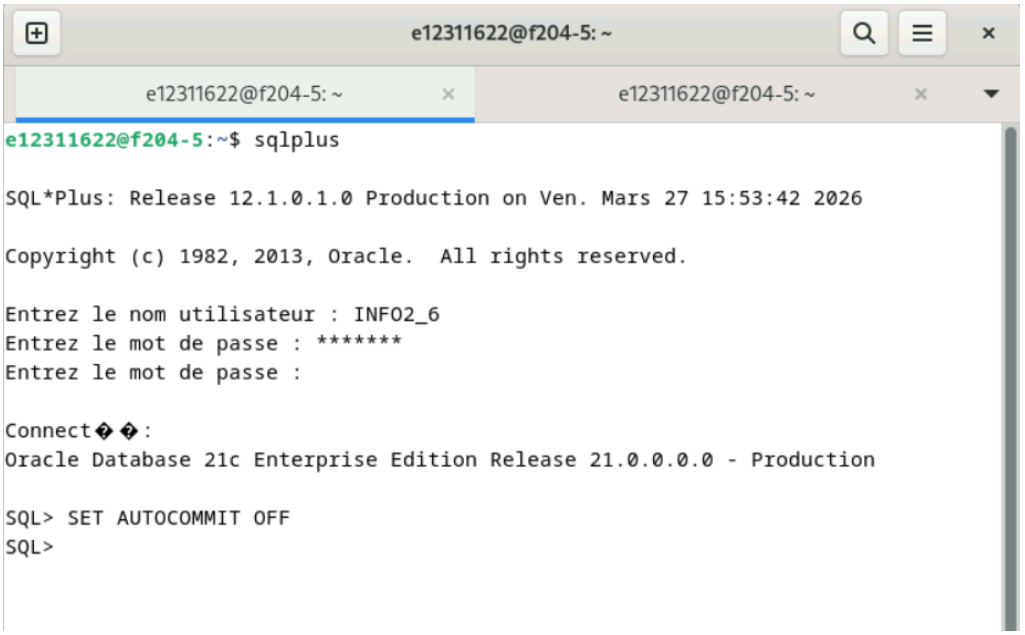
Matière : BDA
Enseignant : Samir Youcef

TP5 : Transactions et contrôle de concurrence

Exercice 1 - Atomicité d'une transaction

Ouvrir les deux sessions

Pour commencer, on ouvre deux terminaux. On considère la première session comme S1 et la seconde comme S2. Dans chacune des sessions, on se connecte et on exécute la commande suivante : ***SET AUTOCOMMIT OFF*** Cette commande doit être exécutée à chaque nouvelle connexion. Elle permet de s'assurer qu'aucune opération ne sera validée automatiquement.



```
e12311622@f204-5: ~
e12311622@f204-5: ~        e12311622@f204-5: ~
e12311622@f204-5:~$ sqlplus

SQL*Plus: Release 12.1.0.1.0 Production on Ven. Mars 27 15:57:59 2026

Copyright (c) 1982, 2013, Oracle.  All rights reserved.

Entrez le nom utilisateur : INFO2_6
Entrez le mot de passe : *******
Entrez le mot de passe :
Heure de la derni►re connexion r►ussie : Ven. Mars  27 2026 15:53:56 +01:00

Connect►►:
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production

SQL> SET AUTOCOMMIT OFF
SQL> ▌
```

Question 1 - créer la table dans S1

Dans la session S1, on saisit le script de création de la table :

```
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production

SQL> SET AUTOCOMMIT OFF
SQL> CREATE TABLE transaction(
  2      idTransaction VARCHAR2(44),
  3      valTransaction NUMBER(10)
  4  );

Table cr►►.

SQL>
```

```
SQL> DESC transaction;
 Nom                                       NULL ?   Type
 ----------------------------------------- -------- ----------------------------
 IDTRANSACTION                                      VARCHAR2(44)
 VALTRANSACTION                                     NUMBER(10)
```

Question 2 - insert / update / delete / rollback dans S2

Dans la session S2, on effectue des opérations de manipulation des données, par exemple des insertions, des modifications et des suppressions.

```
SQL> SET AUTOCOMMIT OFF
SQL> INSERT INTO transaction VALUES ('T1', 100);
INSERT INTO transaction VALUES ('T2', 200);
INSERT INTO transaction VALUES ('T3', 300);


1 ligne cr►►.

SQL>
1 ligne cr►►.

SQL>
1 ligne cr►►.

SQL> SQL>
```

```
SQL> SQL> UPDATE transaction
  2  SET valTransaction = 150
  3  WHERE idTransaction = 'T1';

1 ligne mise ►jour.

SQL> DELETE FROM transaction
  2  WHERE idTransaction = 'T2';

1 ligne supprim►.

SQL>
```

On consulte ensuite le contenu de la table avant d'annuler :

```
SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T1                                                      150
T3                                                      300

SQL>
```

On annule toutes les mises à jour en cours avec la commande ***ROLLBACK*** :

```
SQL> ROLLBACK;

Annulation (rollback) effectu►.

SQL>
```

On vérifie à nouveau le contenu de la table :

```
SQL> SELECT * FROM transaction;

aucune ligne s►lectionn►e

SQL>
```

Toutes les modifications effectuées depuis le début de la transaction sont annulées. La table revient ainsi à son état précédent.

Question 3 - insérer puis quitter avec quit

Toujours dans la session S2, on insère de nouvelles lignes :

```
SQL> INSERT INTO transaction VALUES ('T4', 400);

1 ligne cr►►e.

SQL> INSERT INTO transaction VALUES ('T5', 500);

1 ligne cr►►e.
```

On vérifie immédiatement le contenu dans S2 :

```
SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T4                                                      400
T5                                                      500
```

On ferme ensuite la session S2 :

```
SQL> quit
D►connect►de Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Produc
tion
e12311622@f204-5:~$
```

On consulte le contenu de la table dans la session S1 :

```
SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T4                                                      400
T5                                                      500

SQL>
```

L'exécution de la commande ***quit*** (fermeture normale de la session) a pour effet de valider (COMMIT implicite) la transaction en cours.

Question 4 - fermeture brutale

Dans la session S1, on insère quelques lignes, puis on vérifie l'insertion :

```
SQL> INSERT INTO transaction VALUES ('T6', 600);

1 ligne cr.

SQL> INSERT INTO transaction VALUES ('T7', 700);

1 ligne cr.

SQL>
```

```
SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T4                                                      400
T5                                                      500
T6                                                      600
T7                                                      700

SQL>
```

Puis on ferme **brutalement** la fenêtre SQL*Plus :.

On se reconnecte sur le compte, puis on affiche le contenu de la table. On vérifie si les données saisies ont été préservées :

```
Connect►►:
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production

SQL> SELECT * FROM transaction;

IDTRANSACTION                                VALTRANSACTION
-------------------------------------------- --------------
T4                                                      400
T5                                                      500

SQL> ▌
```

Une fermeture brutale annule également la transaction non validée.

Question 5 - modification de structure puis rollback

On se reconnecte. On insère quelques lignes :

```
SQL> SET AUTOCOMMIT OFF
SQL>
```

Ensuite :

```
SQL> INSERT INTO transaction VALUES ('T8', 800);

1 ligne cr►►.

SQL> INSERT INTO transaction VALUES ('T9', 900);

1 ligne cr►►.

SQL> ▌
```

On modifie ensuite la structure de la table :

```
SQL> ALTER TABLE transaction
  2  ADD val2transaction NUMBER(10);

Table modifi►.

SQL>
```

On tente d'exécuter la commande ***ROLLBACK*** :

```
SQL> ROLLBACK;

Annulation (rollback) effectu.

SQL>
```

Puis on vérifie :

```
SQL> SELECT * FROM transaction;
DESC transaction;

IDTRANSACTION                                VALTRANSACTION VAL2TRANSACTION
-------------------------------------------- -------------- ---------------
T4                                                      400
T5                                                      500
T8                                                      800
T9                                                      900

SQL>  Nom                                          NULL ?   Type
 ----------------------------------------- -------- ----------------------------
 IDTRANSACTION                                       VARCHAR2(44)
 VALTRANSACTION                                      NUMBER(10)
 VAL2TRANSACTION                                     NUMBER(10)

SQL>
```

Sous Oracle, une commande ***ALTER TABLE*** provoque un **COMMIT implicite**, ce qui valide automatiquement la transaction non validée en cours.

Question 6 - Conclusion

Une session correspond à une connexion ouverte entre un utilisateur et le système de gestion de base de données. Tant que cette connexion reste active, l'utilisateur peut exécuter des commandes et interagir avec la base.

Une transaction est un ensemble d'opérations effectuées sur la base de données et considérées comme une seule unité logique de travail. Ces opérations doivent être traitées globalement.

La commande COMMIT permet de valider définitivement les modifications réalisées pendant la transaction. Une fois le COMMIT exécuté, les changements sont enregistrés de manière permanente dans la base de données.

La commande ROLLBACK permet d'annuler toutes les modifications non encore validées de la transaction en cours. La base de données revient alors à son état précédent.

Le principe d'atomicité signifie qu'une transaction est exécutée entièrement ou pas du tout : soit toutes les opérations sont validées, soit aucune n'est conservée.

Exercice 2 - Transactions concurrentes

1. Création des tables

On crée le schéma de la base de données de gestion des billets d'avions, qui est constitué des relations Client et Vol :

```
SQL>
SQL> CREATE TABLE vol(
  2      idVol VARCHAR2(44),
  3      capaciteVol NUMBER(10),
  4      nbrPlacesReserveesVol NUMBER(10)
  5  );

Table cr??.

SQL> CREATE TABLE client(
  2      idClient VARCHAR2(44),
  3      prenomClient VARCHAR2(11),
  4      nbrPlacesReserveesCleint NUMBER(10)
  5  );

Table cr??.
```

Puis on insère un vol et deux clients :

```
SQL> INSERT INTO vol VALUES ('V1', 100, 0);

INSERT INTO client VALUES ('C1', 'Ali', 0);
INSERT INTO client VALUES ('C2', 'Sara', 0);

COMMIT;

1 ligne cr??.

SQL> SQL>
1 ligne cr??.

SQL>
1 ligne cr??.

SQL> SQL>
Validation effectu?.
```

On vérifie les insertions :

```
SQL> SELECT * FROM vol;
SELECT * FROM client;

IDVOL                                      CAPACITEVOL NBRPLACESRESERVEESVOL
------------------------------------------ ----------- ---------------------
V1                                                 100                     0

SQL>
IDCLIENT                                   PRENOMCLIEN
------------------------------------------ -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                         Ali
                      0

C2                                         Sara
                      0
```

2. Isolation des transactions

On exécute deux transactions **T1 (dans S1) et T2 (dans S2)** en parallèle.

Dans S1 (T1) : On effectue une réservation pour le client C1 sans valider la transaction. On affiche le contenu de la table. On constate que les mises à jour sont visibles par T1.

```
SQL> UPDATE client
  2  SET nbrPlacesReserveesCleint = nbrPlacesReserveesCleint + 2
  3  WHERE idClient = 'C1';

1 ligne mise ⋄jour.
```

```
SQL> UPDATE vol
  2  SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 2
  3  WHERE idVol = 'V1';

1 ligne mise ⋄jour.
```

```
SQL> SELECT * FROM client;

IDCLIENT                                   PRENOMCLIEN
------------------------------------------ -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                         Ali
                      2

C2                                         Sara
                      0


SQL> SELECT * FROM vol;

IDVOL                                      CAPACITEVOL NBRPLACESRESERVEESVOL
------------------------------------------ ------------ ---------------------
V1                                                 100                     2

SQL>
```

Dans S2 (T2) : On affiche le contenu de la table. On constate qu'elles s'affichent également dans S2.

```
SQL> SELECT * FROM client;

IDCLIENT                                       PRENOMCLIEN
---------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                             Ali
                       0

C2                                             Sara
                       0


SQL> SELECT * FROM vol;

IDVOL                                          CAPACITEVOL NBRPLACESRESERVEESVOL
---------------------------------------------- ----------- ---------------------
V1                                                     100                     0

SQL>
```

3. Tester ROLLBACK

Dans S1, on exécute la commande ***ROLLBACK***.

```
SQL> ROLLBACK;

Annulation (rollback) effectu.

SQL>
```

Puis on vérifie dans **S1** :

```
SQL> SELECT * FROM client;

IDCLIENT                                       PRENOMCLIEN
---------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                             Ali
                       0

C2                                             Sara
                       0


SQL> SELECT * FROM vol;

IDVOL                                          CAPACITEVOL NBRPLACESRESERVEESVOL
---------------------------------------------- ----------- ---------------------
V1                                                     100                     0

SQL>
```

Et dans **S2** :

```
SQL> SELECT * FROM client;

IDCLIENT                                           PRENOMCLIEN
-------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                                 Ali
                       0

C2                                                 Sara
                       0


SQL> SELECT * FROM vol;

IDVOL                                              CAPACITEVOL NBRPLACESRESERVEESVOL
-------------------------------------------- ----------- ---------------------
V1                                                     100                     0

SQL>
```

Pour la transaction T2, l'opération se déroule comme si T1 n'avait pas existé. La base de données revient à son état initial.

4. Tester COMMIT

Dans S1, on recommence la transaction (la réservation) :

```
SQL> UPDATE client
  2  SET nbrPlacesReserveesCleint = nbrPlacesReserveesCleint + 2
  3  WHERE idClient = 'C1';

1 ligne mise ►jour.

SQL>
SQL> UPDATE vol
  2  SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 2
  3  WHERE idVol = 'V1';

1 ligne mise ►jour.
```

Puis, on la valide avec la commande ***COMMIT*** :

```
SQL> COMMIT;

Validation effectu►e.
```

Dans S2, on affiche le contenu de la table :

```
SQL> SELECT * FROM client;

IDCLIENT                                           PRENOMCLIEN
-------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                                 Ali
                       2

C2                                                 Sara
                       0


SQL> SELECT * FROM vol;

IDVOL                                              CAPACITEVOL NBRPLACESRESERVEESVOL
-------------------------------------------- ----------- ---------------------
V1                                                     100                     2

SQL> ▌
```

5. Isolation incomplète = incohérence possible

On réinitialise d’abord les données :

Dans S1:

```
SQL> UPDATE client
  2  SET nbrPlacesReserveesCleint = 0;

2 lignes mises ►jour.

SQL>
SQL> UPDATE vol
  2  SET nbrPlacesReserveesVol = 0;

1 ligne mise ►jour.

SQL>
SQL> COMMIT;

Validation effectu►e.

SQL>
```

On vérifie :

Dans S1 :

```
SQL> SELECT * FROM client;

IDCLIENT                                           PRENOMCLIEN
-------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                                 Ali
                       0

C2                                                 Sara
                       0


SQL> SELECT * FROM vol;

IDVOL                                              CAPACITEVOL NBRPLACESRESERVEESVOL
-------------------------------------------- ----------- ---------------------
V1                                                     100                     0

SQL> ▌
```

Dans S2 :

```
SQL> SELECT * FROM client;

IDCLIENT                                           PRENOMCLIEN
-------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                                 Ali
                       0

C2                                                 Sara
                       0


SQL> SELECT * FROM vol;

IDVOL                                              CAPACITEVOL NBRPLACESRESERVEESVOL
-------------------------------------------- ----------- ---------------------
V1                                                     100                     0

SQL>
```

<u>Étape 1 :</u> lectures de T1 dans S1

```
SQL> SELECT * FROM vol WHERE idVol = 'V1';

IDVOL                                          CAPACITEVOL NBRPLACESRESERVEESVOL
---------------------------------------------- ----------- ---------------------
V1                                                     100                     0

SQL> SELECT * FROM client WHERE idClient = 'C1';

IDCLIENT                                       PRENOMCLIEN
---------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                             Ali
                       0
```

Étape 2 : lectures de T2 dans S2

```
SQL> SELECT * FROM vol WHERE idVol = 'V1';

IDVOL                                          CAPACITEVOL NBRPLACESRESERVEESVOL
---------------------------------------------- ----------- ---------------------
V1                                                     100                     0

SQL> SELECT * FROM client WHERE idClient = 'C2';

IDCLIENT                                       PRENOMCLIEN
---------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C2                                             Sara
                       0
```

Étape 3 : mises à jour de T1 puis COMMIT dans S1

```
SQL> UPDATE client
  2  SET nbrPlacesReserveesCleint = 2
  3  WHERE idClient = 'C1';

1 ligne mise ◆jour.

SQL>
SQL> UPDATE vol
  2  SET nbrPlacesReserveesVol = 2
  3  WHERE idVol = 'V1';

1 ligne mise ◆jour.

SQL>
SQL> COMMIT;

Validation effectu◆.

SQL>
```

<u>Étape 4 :</u> mises à jour de T2 puis COMMIT dans S2

Ici, T2 a lu avant que T1 ne valide. Elle peut donc faire une mise à jour basée sur une ancienne valeur.

```
SQL> UPDATE client
  2  SET nbrPlacesReserveesCleint = 3
  3  WHERE idClient = 'C2';

1 ligne mise ►jour.

SQL>
SQL> UPDATE vol
  2  SET nbrPlacesReserveesVol = 3
  3  WHERE idVol = 'V1';

1 ligne mise ►jour.

SQL>
SQL> COMMIT;

Validation effectu►.

SQL>
```

Vérification finale :

Sur S1 :

```
SQL> SELECT * FROM client;

IDCLIENT                                           PRENOMCLIEN
-------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                                 Ali
                      2

C2                                                 Sara
                      3


SQL> SELECT * FROM vol;

IDVOL                                              CAPACITEVOL NBRPLACESRESERVEESVOL
-------------------------------------------- ----------- ---------------------
V1                                                     100                     3

SQL>
```

Sur S2 :

```
SQL> SELECT * FROM client;

IDCLIENT                                           PRENOMCLIEN
-------------------------------------------- -----------
NBRPLACESRESERVEESCLEINT
------------------------
C1                                                 Ali
                       2

C2                                                 Sara
                       3


SQL> SELECT * FROM vol;

IDVOL                                              CAPACITEVOL NBRPLACESRESERVEESVOL
-------------------------------------------- ----------- ---------------------
V1                                                     100                     3

SQL>
```

Après vérification finale, on constate que les **clients** ont réservé **5 places au total (2 + 3)**, mais que la colonne **vol.nbrPlacesReserveesVol** peut valoir **seulement 3**.

La base de données devient **incohérente**. On a reproduit un problème de **mise à jour perdue** dû à un niveau d'isolation incomplet.